Name: Parth Pankaj Thikrul

PRN: 25070122204

Batch: CSE C1

EXPERIMENT 12

To understand the basic use of Git, Jenkins, and Apache Tomcat for managing and running a web application.

Course Outcome	CO5	Teaching Hours	2

Objective

- To understand the basic concept and use of **Git** for version control.
- To understand how to upload and manage a web application using **GitHub**.
- To understand the role of **Jenkins** in building and automating the application.
- To understand the use of **Apache Tomcat** for running a web application.
- To understand the basic flow: **GitHub → Jenkins → Tomcat → Web Application**.

Theory

Git is a distributed version control system used to track changes in source-code files. It allows developers to maintain different versions of a project and upload the code to platforms such as GitHub. GitHub is an online platform that hosts Git repositories. It allows users to store, manage, and share their source code. Jenkins is an automation server used to automate tasks such as fetching source code, building applications, and deploying applications. Jenkins can automatically perform these tasks whenever changes are made to the source code. Apache Tomcat is a web server and servlet container used to run Java-based web applications. It receives requests from a browser and serves the web application.

Software / Hardware Requirements

Software	Purpose
Windows 10/11	Operating system
Git	Version control
GitHub Account	Online code repository
Jenkins	Automation and build server
Apache Tomcat	Web application server
Java JDK	Required for Jenkins and Tomcat
Web Browser	To access the application
VS Code / Notepad++	To create/edit HTML files

Step-by-Step Procedure

Step 1: Create a Web Application

Create a simple HTML file named index.html.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Web Application</title>
</head>
<body>
  <h1>Hello from Git, Jenkins and Tomcat!</h1>
  <p>This is my first web application.</p>
</body>
</html>
```

Create a project folder, for example:

```
MyWebApp
  └── index.html
```

Step 2: Initialize Git

Open Command Prompt/Terminal inside the project folder and execute:

```
git init
```

Add the project files:

```
git add .
```

Create the first commit:

```
git commit -m "Initial commit"
```

Step 3: Create a GitHub Repository

1. Log in to GitHub.
2. Create a new repository.
3. Give it a suitable name, such as MyWebApp.
4. Copy the repository URL.

Connect the local project to GitHub:

```
git remote add origin <GitHub-Repository-URL>
```

Push the project:

git branch -M main

git push -u origin main

The web application files should now be visible in the GitHub repository.

Step 4: Install and Start Jenkins

1. Install **Jenkins**.
2. Open Jenkins in a web browser:

http://localhost:8080

3. Complete the initial Jenkins setup.
4. Create a Jenkins job, for example:

MyWebApp

Step 5: Configure Jenkins

In the Jenkins project:

1. Select **Pipeline** or **Freestyle Project**.
2. Configure the GitHub repository URL.
3. Select the required Git branch, usually:

main

4. Configure the build step.
5. Save the Jenkins configuration.
6. Click **Build Now**.

Jenkins will fetch the source code from GitHub and execute the configured build steps.

Step 6: Install and Configure Apache Tomcat

1. Download and extract Apache Tomcat.
2. Configure the required Java environment.
3. Start Tomcat.
4. If port 8080 is already being used by Jenkins, configure Tomcat to use another port, for example:

8081

5. Start the Tomcat server.

Open:

http://localhost:8081

The Tomcat welcome page should appear.

Step 7: Deploy the Web Application

Copy the web application files into the appropriate Tomcat deployment directory, such as:

apache-tomcat

└── webapps

└── MyWebApp

└── index.html

Start/restart Tomcat if required.

Open the application in the browser:

http://localhost:8081/MyWebApp/

The web application should be displayed.

Step 8: Test the Workflow

Modify index.html, for example:

<h1>Updated Web Application</h1>

Commit and push the changes:

git add .

git commit -m "Updated web application"

git push

Run the Jenkins job again.

Jenkins fetches the updated code and performs the configured build/deployment process. After deployment, refresh the application in the browser to verify the changes.

Output Screenshots

```
C:\Users\mitta>java --version
java 26.0.2 2026-07-21
Java(TM) SE Runtime Environment (build 26.0.2+10-55)
Java HotSpot(TM) 64-Bit Server VM (build 26.0.2+10-55, mixed mode, sharing)

C:\Users\mitta>git --version
git version 2.56.0.windows.2

C:\Users\mitta>where git
C:\Program Files\Git\cmd\git.exe

C:\Users\mitta>where java
C:\Program Files\Common Files\Oracle\Java\javapath\java.exe
C:\Program Files (x86)\Common Files\Oracle\Java\java8path\java.exe
C:\Program Files\Java\jdk-26.0.2\bin\java.exe
```

Figure 1: Software verification – *Java and Git versions in Command Prompt*

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My Web Application</title>
</head>
<body>
    <h1>Hello from Git, Jenkins and Tomcat!</h1>
    <p>This is my first web application.</p>
    <p>Version 1</p>
</body>
</html>
```

Figure 2: HTML webpage – *index.html code in VS Code*

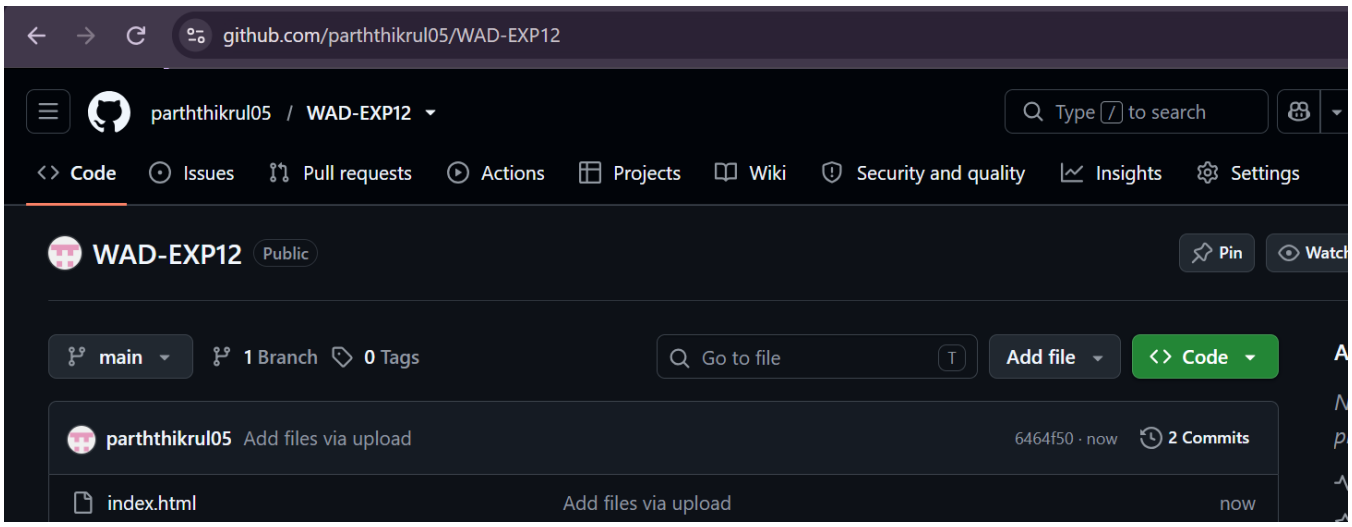


Figure 3: GitHub repository – *index.html on the main branch*

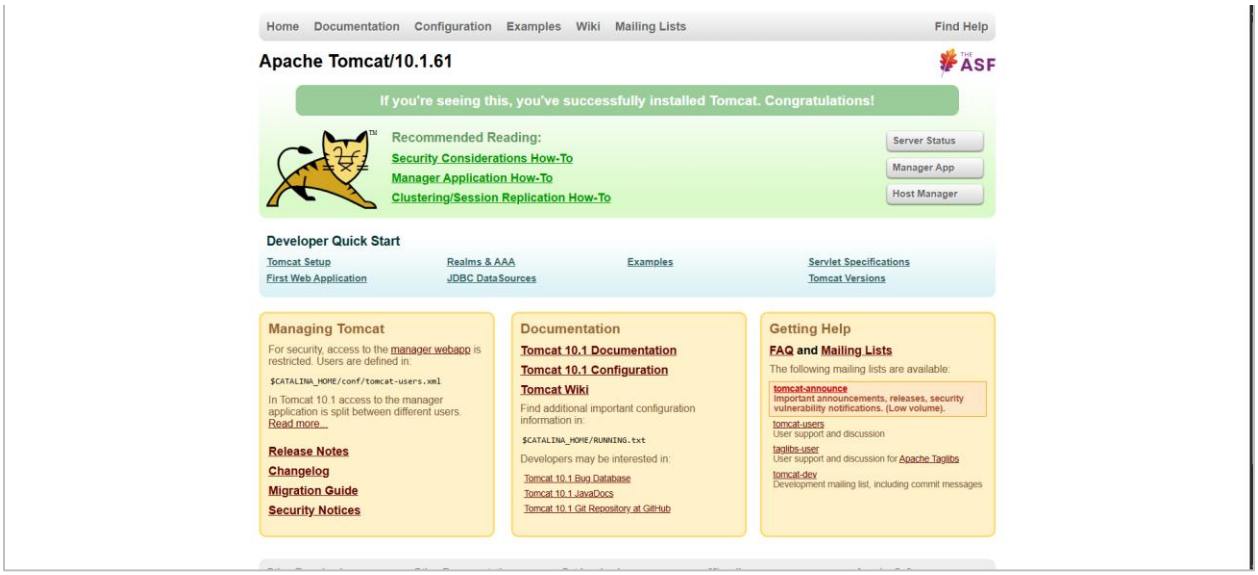


Figure 4: Tomcat running – *Tomcat welcome page*

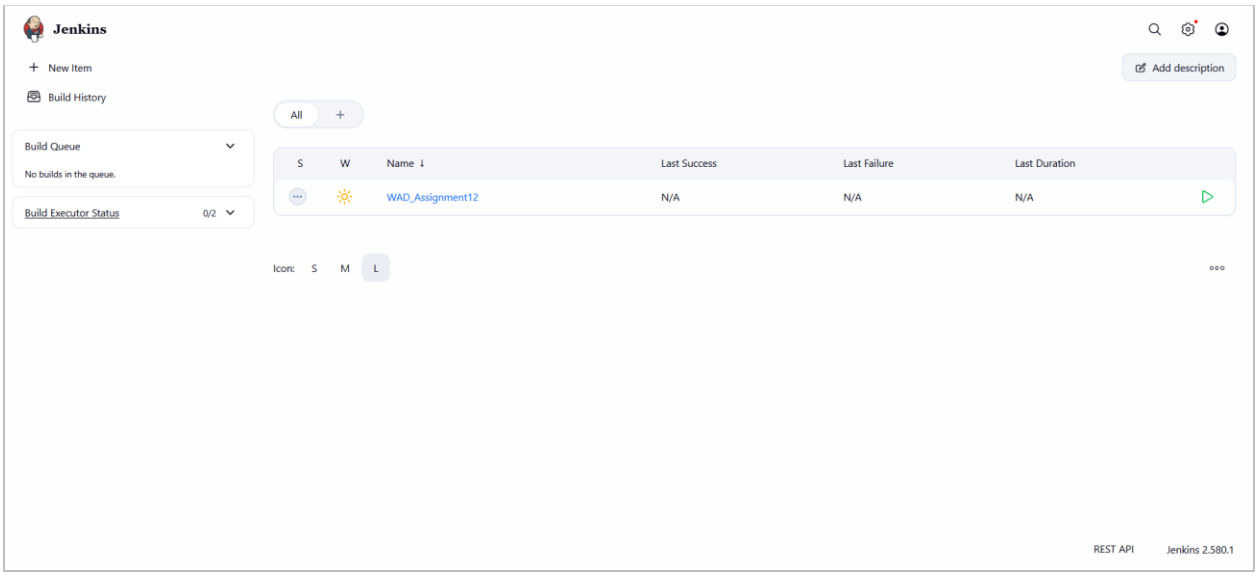


Figure 5: Jenkins dashboard – *Jenkins setup completed*

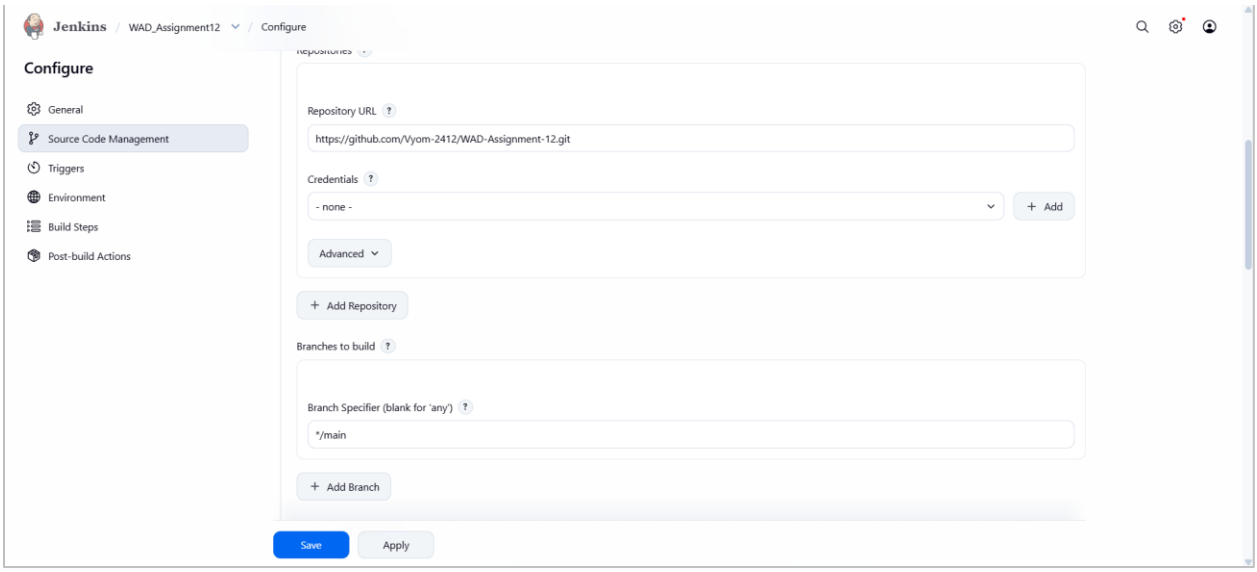


Figure 6: Jenkins configuration – *Git repository URL and branch (*/main)*



Figure 7: Jenkins configuration – *build step with the deployment command*



Figure 8: Successful first build – *Console Output showing Finished: SUCCESS*

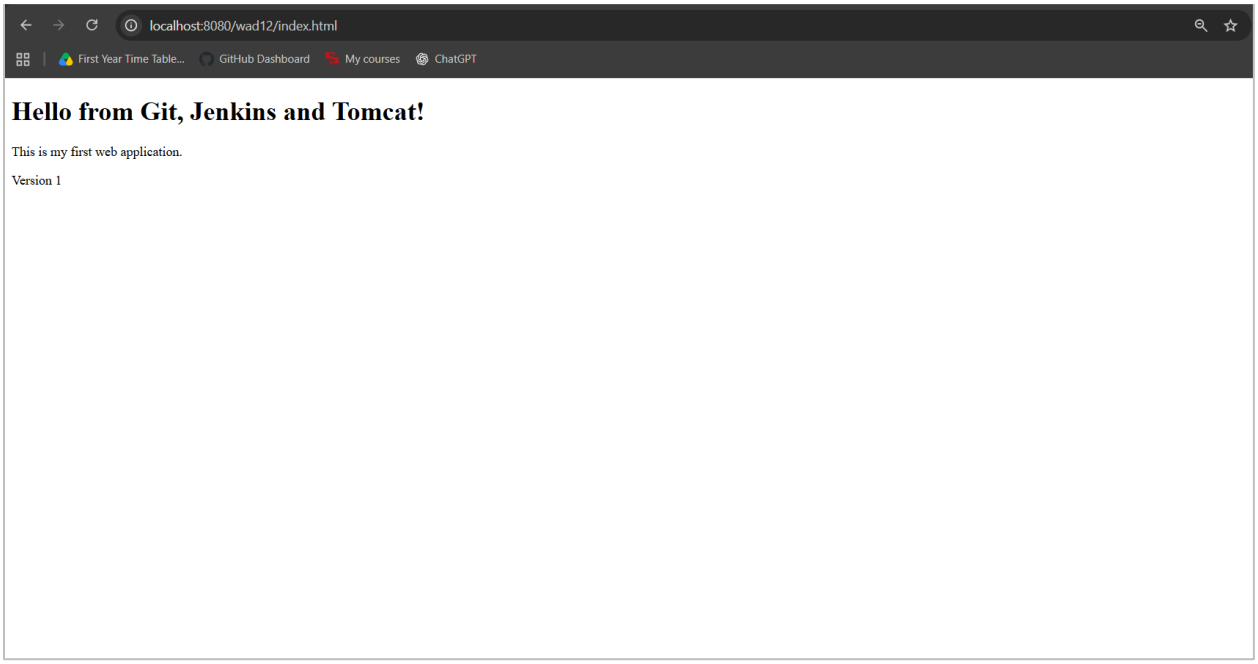


Figure 9: Version 1 deployment – *original webpage in the browser*

```
PS C:\Users\mitta\OneDrive\Desktop\test\assign12_wad> git init
Initialized empty Git repository in C:/Users/mitta/OneDrive/Desktop/test/assign12_wad/.git/
PS C:\Users\mitta\OneDrive\Desktop\test\assign12_wad> git add .
PS C:\Users\mitta\OneDrive\Desktop\test\assign12_wad> git commit -m "initial commit"
[master (root-commit) 707a602] initial commit
 1 file changed, 12 insertions(+)
 create mode 100644 index.html
PS C:\Users\mitta\OneDrive\Desktop\test\assign12_wad> git branch -M main
PS C:\Users\mitta\OneDrive\Desktop\test\assign12_wad> git remote add origin https://github.com/Vyom-2412/WAD-Assignment-12.git
PS C:\Users\mitta\OneDrive\Desktop\test\assign12_wad> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 393 bytes | 393.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Vyom-2412/WAD-Assignment-12.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\mitta\OneDrive\Desktop\test\assign12_wad> git add index.html
PS C:\Users\mitta\OneDrive\Desktop\test\assign12_wad> git commit -m "Update webpage to Version 2"
[main d67bd40] Update webpage to Version 2
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\mitta\OneDrive\Desktop\test\assign12_wad> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 303 bytes | 303.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Vyom-2412/WAD-Assignment-12.git
   707a602..d67bd40  main -> main
```

Figure 10: Version 2 update – *updated code committed and pushed to GitHub*

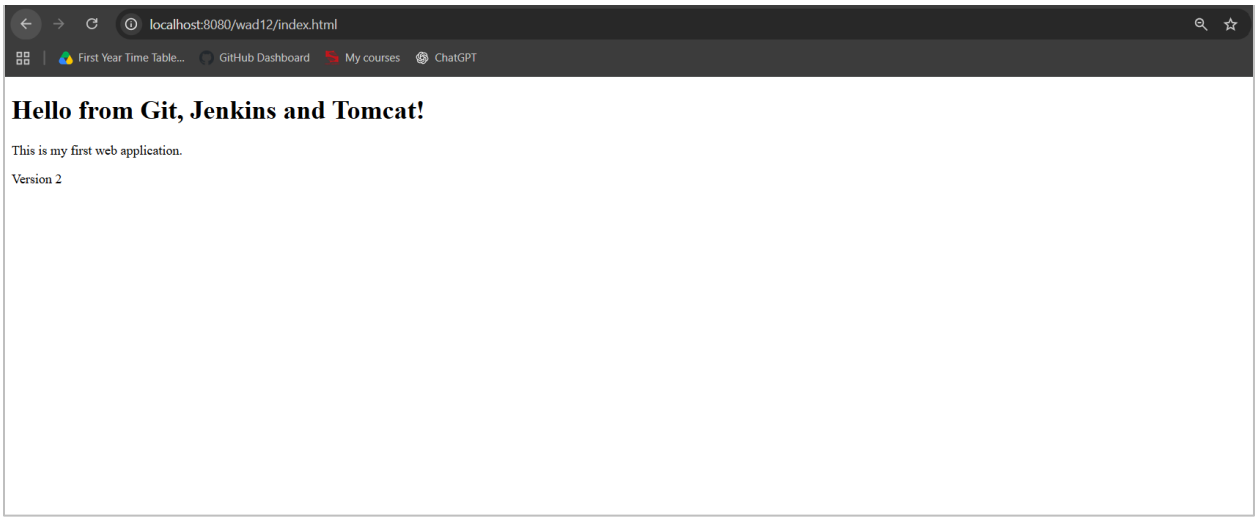


Figure 11: Version 2 update – *refreshed webpage after the second build*

Conclusion

This experiment provides a basic understanding of version control, automation, and web application deployment. Git and GitHub are used to manage and store the application source code, Jenkins automates the build and deployment process, and Apache Tomcat provides the environment for running the web application. Thus, the basic principles of a simple CI/CD workflow are understood.